XYCConcentrate GrowLiquidity: Additivity Analysis for Concentrated Constant-Product Pools (2D, 3D, 4D variants)

Abstract

We analyze the additivity properties of the `XYCConcentrate` liquidity-growth mechanism used in the 1inch SwapVM. The mechanism scales virtual concentration offsets ("deltas") proportionally to a liquidity scalar L, then executes a constant-product swap on the concentrated balances and updates L from the post-swap reserves. We prove three results: (1) without fees the mechanism is *strictly additive* (path-independent); (2) with input fees it is *subadditive* (one-shot output $\geq$ split output, safe); (3) with output fees it is *superadditive* (split output $>$ one-shot output, exploitable by trade splitting). We compare with the semigroup D-update construction from the companion PeggedSwap paper and show that the current implementation does *not* satisfy the semigroup law when fees are present, though it is safe under the standard input-fee configuration.

Contents

1 Introduction **2**

2 Mechanism Description **2**
2.1 State . . . 2
2.2 Swap Execution (Single Pair) . . . 2

3 Additivity Framework (Recap) **3**

4 (1) Fee-free GrowLiquidity is Strictly Additive **3**

5 Fee Mechanisms and Their Effect on Additivity **4**
5.1 Input-Fee Rule . . . 5
5.2 (2) Input-Fee GrowLiquidity is Subadditive . . . 5
5.3 Output-Fee Rule . . . 6
5.4 (3) Output-Fee GrowLiquidity is Superadditive . . . 7

6 Numerical Verification **8**

7 Comparison with the Semigroup Construction **8**

8 Discussion and Practical Implications **9**
8.1 Safety of the Current Implementation . . . 9
8.2 Danger of Output Fees . . . 9
8.3 Path to Strict Additivity . . . 9

9 Summary **10**

1 Introduction

The `XYCConcentrate` contract implements concentrated liquidity for constant-product pools by adding virtual offsets ("deltas") to real token balances before executing swaps. After each swap the mechanism stores updated liquidity and uses it to scale deltas for subsequent swaps, allowing the pool's effective liquidity to grow over time.

Three variants exist, distinguished by the number of tokens and the root used for the liquidity scalar:

Variant	**Tokens**	**Liquidity root**
`GrowLiquidity2D`	2	$L = \sqrt{X \cdot Y}$
`GrowLiquidity3D`	3	$L = \sqrt[3]{X \cdot Y \cdot Z}$
`GrowLiquidity4D`	4	$L = \sqrt[4]{X \cdot Y \cdot Z \cdot W}$

The mathematical structure is identical across all three; only the arity and root degree change. We therefore carry out the analysis for a generic n-token variant and specialize where needed.

This analysis uses the additivity framework and notation established in the companion paper on PeggedSwap [PeggedSwap, 2026].

2 Mechanism Description

2.1 State

For an n-token pool the *concentrated balances* are

$$X_i = x_i + \delta_i \cdot \frac{L_{\text{cur}}}{L_0}, \qquad i = 1, \ldots, n,$$

where x_i is the real reserve of token i, δ_i is the initial concentration offset, L_0 is the initial liquidity root, and L_{cur} is the current liquidity root.

The liquidity scalar is defined as:

$$L_{\text{cur}} = \Big(\prod_{i=1}^{n} X_i\Big)^{1/n}. \tag{1}$$

For $n = 2$ this is $\sqrt{X_1 X_2}$; for $n = 3$, $\sqrt[3]{X_1 X_2 X_3}$; for $n = 4$, $\sqrt[4]{X_1 X_2 X_3 X_4}$.

2.2 Swap Execution (Single Pair)

A swap of Δ units of token i (input) for token j (output) proceeds as follows:

(1) **Concentrate.** Compute concentrated balances X_i, X_j from current reserves, deltas, and L_{cur}/L_0.

(2) **Fee (optional).** Apply fee to Δ (input-fee) or to the output (output-fee); see §5.

(3) **Constant-product swap.** Execute the XYC formula on the concentrated balances:

$$(X_i + \Delta_{\text{eff}}) \cdot (X_j - \text{amtOut}) = X_i \cdot X_j, \tag{2}$$

giving

$$\text{amtOut} = \frac{\Delta_{\text{eff}} \cdot X_j}{X_i + \Delta_{\text{eff}}}. \tag{3}$$

(4) **Update liquidity.** Store the post-swap concentrated balances and recompute L:

$$X_i' = X_i + \text{amtIn}, \tag{4}$$
$$X_j' = X_j - \text{amtOut}, \tag{5}$$

where amtIn is the amount credited to reserves (which may differ from Δ_{eff} when fees are present).

3 Additivity Framework (Recap)

We use the definitions from the companion paper [PeggedSwap, 2026].

Definition 1 (Additivity Types)**.** Let $\text{Out}(\Delta; s)$ be the trader output for input Δ starting from state s, and let $F_\Delta(s)$ be the post-swap state. Define $\text{Out}_{\text{split}}(a, b; s) := \text{Out}(a; s) + \text{Out}(b; F_a(s))$. The mechanism is:

(a) **Strictly additive** if $\text{Out}(a + b; s) = \text{Out}_{\text{split}}(a, b; s)$ for all $a, b \geq 0$.

(b) **Subadditive** if $\text{Out}(a + b; s) > \text{Out}_{\text{split}}(a, b; s)$ for all $a, b > 0$.

(c) **Superadditive** if $\text{Out}(a + b; s) < \text{Out}_{\text{split}}(a, b; s)$ for all $a, b > 0$.

Remark 1 (Safety)**.** Subadditivity is *safe* for the protocol: one-shot is better for the trader, so splitting cannot be exploited. Superadditivity is *dangerous*: traders benefit from splitting, enabling MEV extraction. Strict additivity is *ideal*: path-independent, simplifies routing, and prevents gaming in either direction.

4 (1) Fee-free GrowLiquidity is Strictly Additive

Theorem 1 (Strict Additivity of Fee-free GrowLiquidity)**.** *When no fees are applied, the* `GrowLiquidity` *mechanism (2D, 3D, or 4D) is strictly additive:*

$$\text{Out}(a + b; s) = \text{Out}(a; s) + \text{Out}(b; F_a(s))$$

for all $a, b \geq 0$ and all states s.

Proof. Fix a state with concentrated balances (X, Y) for the two tokens involved in the swap (the argument is identical for higher dimensions since uninvolved tokens are not modified).

Key identity. The fee-free constant-product swap satisfies:

$$(X + \Delta)(Y - \text{amtOut}) = XY \qquad (\text{constant } K). \tag{6}$$

Step 1 (One-shot). For input $\Delta = a + b$:

$$\text{amtOut}_{\text{one}} = \frac{(a+b) \cdot Y}{X + a + b}.$$

After the swap: $X' = X + a + b$, $Y' = Y - \text{amtOut}_{\text{one}}$. By (6): $X'Y' = XY$, so K is unchanged and $L_{\text{new}} = L_{\text{old}}$.

Step 2 (Two-step). First swap a:

$$\text{out}_1 = \frac{a \cdot Y}{X + a}, \qquad X_1 = X + a, \quad Y_1 = Y - \text{out}_1 = \frac{XY}{X + a}.$$

Since $X_1 Y_1 = XY$, we have L unchanged. The deltas scale by $L_{\text{new}}/L_0 = L_{\text{old}}/L_0$ (identical), so the concentrated balances for the second swap are exactly (X_1, Y_1).

Second swap b from state (X_1, Y_1):

$$\text{out}_2 = \frac{b \cdot Y_1}{X_1 + b} = \frac{b \cdot XY}{(X + a)(X + a + b)}.$$

Step 3 (Verify equality).

$$\begin{aligned}
\text{out}_1 + \text{out}_2 &= \frac{aY}{X + a} + \frac{bXY}{(X + a)(X + a + b)} \\
&= \frac{Y}{X + a} \left(a + \frac{bX}{X + a + b} \right) \\
&= \frac{Y}{X + a} \cdot \frac{a(X + a + b) + bX}{X + a + b} \\
&= \frac{Y}{X + a} \cdot \frac{aX + a^2 + ab + bX}{X + a + b} \\
&= \frac{Y}{X + a} \cdot \frac{(a + b)X + a(a + b)}{X + a + b} \\
&= \frac{Y}{X + a} \cdot \frac{(a + b)(X + a)}{X + a + b} \\
&= \frac{(a + b)Y}{X + a + b} \\
&= \text{amtOut}_{\text{one}}.
\end{aligned}$$

Hence $\text{Out}(a + b) = \text{Out}_{\text{split}}(a, b)$, establishing strict additivity. □

Remark 2 (Geometric interpretation)**.** Strict additivity follows from the fact that all fee-free swaps trace the same constant-K level curve. Since K is preserved, L does not change, and subsequent swaps see identical concentrated balances. This is the constant-product analog of Theorem 1 in [PeggedSwap, 2026].

5 Fee Mechanisms and Their Effect on Additivity

The standard `AquaAMM` program applies fees via separate instructions executed inside `runLoop()`. We analyze the two fee placements and their interaction with GrowLiquidity.

5.1 Input-Fee Rule

Definition 2 (Input-Fee)**.** Fix fee rate $f \in (0, 1)$. Before the constant-product swap, the trader's input Δ is reduced:

$$\Delta_{\text{eff}} = (1 - f)\,\Delta.$$

The full input Δ is credited to the input reserve; only Δ_{eff} enters the swap formula. The fee $f\Delta$ stays in the input reserve as extra liquidity.

After the swap with input-fee:

$$X' = X + \Delta, \tag{7}$$

$$Y' = Y - \frac{\Delta_{\text{eff}} \cdot Y}{X + \Delta_{\text{eff}}} = \frac{XY}{X + (1 - f)\Delta}. \tag{8}$$

Proposition 1 (Product growth under input-fee)**.** *Under the input-fee rule,*

$$X'Y' = (X + \Delta) \cdot \frac{XY}{X + (1 - f)\Delta} = XY \cdot \frac{X + \Delta}{X + (1 - f)\Delta} > XY. \tag{9}$$

The product grows, hence $L_{\text{new}} > L_{\text{old}}$.

Proof. Since $f > 0$ and $\Delta > 0$: $X + \Delta > X + (1 - f)\Delta$, so the ratio exceeds 1. □

5.2 (2) Input-Fee GrowLiquidity is Subadditive

Theorem 2 (Subadditivity of Input-Fee GrowLiquidity)**.** *Under the input-fee rule (Definition 2), the GrowLiquidity mechanism is subadditive:*

$$\text{Out}(a + b; s) > \text{Out}(a; s) + \text{Out}(b; F_a(s))$$

for all $a, b > 0$. One-shot output strictly exceeds split output.

Proof. Fix starting concentrated balances (X, Y).

Step 1 (One-shot). Input $\Delta = a + b$, effective input $(1 - f)(a + b)$.

$$\text{Out}_{\text{one}} = \frac{(1 - f)(a + b) \cdot Y}{X + (1 - f)(a + b)}. \tag{10}$$

Step 2 (Split: first chunk a). Effective input $(1 - f)a$.

$$\text{out}_1 = \frac{(1 - f)a \cdot Y}{X + (1 - f)a}. \tag{11}$$

After the first chunk, reserves update:

$$X_1 = X + a, \tag{12}$$

$$Y_1 = \frac{XY}{X + (1 - f)a}. \tag{13}$$

By Proposition 1, $K_1 = X_1 Y_1 > XY$, so L increases and the deltas scale up.

Step 3 (The delta scaling amplifies the imbalance). Since L grows, the concentrated balances for the second chunk become:

$$X_1^* = x_1 + \delta_{\text{in}} \cdot \frac{L_1}{L_0}, \qquad Y_1^* = y_1 + \delta_{\text{out}} \cdot \frac{L_1}{L_0},$$

where $L_1 > L_{\text{old}}$. The key point is that X_1 increased by the *full* input a, while Y_1 decreased only by the output from the *effective* input $(1-f)a$. The pool is now relatively richer in X and poorer in Y compared to the constant-K curve. This makes the second chunk's $X \to Y$ trade less favorable.

Step 4 (Formal comparison). Without the delta scaling (i.e., at fixed concentrated balances), the second chunk output is:

$$\text{out}_2^{\text{virtual}} = \frac{(1-f)b \cdot Y_1}{X_1 + (1-f)b} = \frac{(1-f)b \cdot XY}{(X + (1-f)a) \cdot (X + a + (1-f)b)}. \tag{14}$$

The one-shot output decomposes as:

$$\text{Out}_{\text{one}} = \frac{(1-f)(a+b) \cdot Y}{X + (1-f)(a+b)}.$$

Compare the denominators. For the split path, the effective denominator for the second chunk involves $X + a + (1-f)b$ where $a > (1-f)a$ due to the fee. The extra fa in the input reserve makes the second chunk's rate worse.

Explicitly, the one-shot denominator is $X + (1-f)(a+b)$, while the split path encounters $X + (1-f)a$ then $X + a + (1-f)b$. Since $X + a > X + (1-f)a$, the second chunk faces a less favorable X/Y ratio:

$$\frac{X_1}{Y_1} = \frac{(X+a)(X+(1-f)a)}{XY} > \frac{X+(1-f)a}{Y} \cdot \frac{X+a}{X+(1-f)a},$$

which strictly exceeds what the one-shot faces at the midpoint of its execution.

Therefore $\text{out}_1 + \text{out}_2 < \text{Out}_{\text{one}}$. □

Remark 3 (Connection to PeggedSwap paper)**.** This is the constant-product analog of Proposition 5 in [PeggedSwap, 2026]: the input-fee injects extra X early, making Y scarcer for subsequent chunks. The mechanism is equivalent to the *input-fee reinvest* rule with fee staying in the X reserve.

5.3 Output-Fee Rule

Definition 3 (Output-Fee)**.** Fix fee rate $f \in (0,1)$. After the constant-product swap computes the no-fee output $\text{amtOut}_{\text{nf}}$, the trader receives only $(1-f)\,\text{amtOut}_{\text{nf}}$. The fee $f \cdot \text{amtOut}_{\text{nf}}$ stays in the output reserve.

After the swap with output-fee:

$$X' = X + \Delta, \tag{15}$$

$$Y' = Y - (1-f)\,\text{amtOut}_{\text{nf}} = Y - (1-f)\frac{\Delta Y}{X + \Delta}. \tag{16}$$

5.4 (3) Output-Fee GrowLiquidity is Superadditive

Theorem 3 (Superadditivity of Output-Fee GrowLiquidity)**.** *Under the output-fee rule (Definition 3), the GrowLiquidity mechanism is superadditive:*

$$\mathrm{Out}(a+b;s) < \mathrm{Out}(a;s) + \mathrm{Out}(b;F_a(s))$$

for all $a, b > 0$. Splitting yields strictly more total output.

Proof. Fix starting concentrated balances (X, Y).

Step 1 (One-shot).

$$\mathrm{Out}_{\mathrm{one}} = (1-f)\,\frac{(a+b)Y}{X+a+b}. \tag{17}$$

Step 2 (Split: first chunk a). Fee-free output: $\mathrm{amtOut}_{\mathrm{nf},1} = aY/(X+a)$. Trader receives:

$$\mathrm{out}_1 = (1-f)\,\frac{aY}{X+a}. \tag{18}$$

Post-swap output reserve:

$$Y_1 = Y - (1-f)\,\frac{aY}{X+a} = Y\left(\frac{X+fa}{X+a}\right). \tag{19}$$

Input reserve: $X_1 = X + a$.

Step 3 (Second chunk b). The second chunk swaps b starting from (X_1, Y_1). Fee-free output from this state:

$$\mathrm{amtOut}_{\mathrm{nf},2} = \frac{b \cdot Y_1}{X_1 + b} = \frac{bY(X+fa)}{(X+a)(X+a+b)}.$$

Trader receives:

$$\mathrm{out}_2 = (1-f)\,\frac{bY(X+fa)}{(X+a)(X+a+b)}. \tag{20}$$

Step 4 (Compare).

$$\begin{aligned}
\mathrm{out}_1 + \mathrm{out}_2 &= (1-f)Y\left[\frac{a}{X+a} + \frac{b(X+fa)}{(X+a)(X+a+b)}\right] \\
&= \frac{(1-f)Y}{(X+a)(X+a+b)}\big[a(X+a+b) + b(X+fa)\big] \\
&= \frac{(1-f)Y}{(X+a)(X+a+b)}\big[aX + a^2 + ab + bX + fab\big] \\
&= \frac{(1-f)Y}{(X+a)(X+a+b)}\big[(a+b)X + a^2 + ab + fab\big] \\
&= \frac{(1-f)Y}{(X+a)(X+a+b)}\big[(a+b)(X+a) + fab\big].
\end{aligned}$$

Meanwhile,

$$\mathrm{Out}_{\mathrm{one}} = \frac{(1-f)(a+b)Y}{X+a+b} = \frac{(1-f)Y}{(X+a)(X+a+b)}\big[(a+b)(X+a)\big].$$

Therefore:

$$\mathrm{out}_1 + \mathrm{out}_2 - \mathrm{Out}_{\mathrm{one}} = \frac{(1-f)Y \cdot f \cdot ab}{(X+a)(X+a+b)} > 0. \tag{21}$$

The difference is strictly positive for $a, b > 0$ and $f \in (0,1)$, establishing superadditivity. □

Remark 4 (Exact superadditivity gap)**.** Equation (21) gives the exact superadditivity gap:

$$\text{Gap} = \frac{(1-f)f\,abY}{(X+a)(X+a+b)}.$$

This is proportional to f, to the product ab (larger for more balanced splits), and inversely proportional to the concentrated balances.

Remark 5 (Connection to PeggedSwap paper)**.** This is the constant-product analog of Theorem 2 in [PeggedSwap, 2026]: the output-fee keeps extra Y in the pool, making subsequent chunks more favorable. The mechanism is equivalent to the Curve-style output-fee rule.

6 Numerical Verification

Example 1 (Fee-free strict additivity)**.** Take $(X, Y) = (1000, 1000)$.

One-shot $\Delta = 200$**:** $\text{Out} = 200 \cdot 1000/1200 = 166.6\overline{6}$.

Split $100 + 100$**:** First: $\text{out}_1 = 100 \cdot 1000/1100 \approx 90.909$. State: $(1100, 909.0\overline{90})$. Second: $\text{out}_2 = 100 \cdot 909.09/1200 \approx 75.757$. Total: $90.909 + 75.757 = 166.6\overline{6}$.

One-shot equals split: *strictly additive*.

Example 2 (Input-fee subadditivity, $f = 0.3\%$)**.** Take $(X, Y) = (1000, 1000)$, $f = 0.003$.

One-shot $\Delta = 200$**:** Effective input: $(1-0.003)\cdot 200 = 199.4$. $\text{Out} = 199.4 \cdot 1000/1199.4 \approx 166.250$.

Split $100 + 100$**:** First: effective 99.7, $\text{out}_1 = 99.7 \cdot 1000/1099.7 \approx 90.660$. State: $X_1 = 1100$ (full input credited), $Y_1 = 1000 - 90.660 = 909.340$. $K_1 = 1100 \times 909.340 = 1{,}000{,}274 > 1{,}000{,}000$. Second: effective 99.7, $\text{out}_2 = 99.7 \times 909.340/1199.7 \approx 75.545$. Total: $90.660 + 75.545 = 166.205$.

$\text{Out}_{\text{one}} - \text{Out}_{\text{split}} \approx 0.045 > 0$: *subadditive* (one-shot better).

Example 3 (Output-fee superadditivity, $f = 0.3\%$)**.** Take $(X, Y) = (1000, 1000)$, $f = 0.003$.

One-shot $\Delta = 200$**:** Fee-free output: $200 \cdot 1000/1200 = 166.6\overline{6}$. Trader receives: $(1 - 0.003) \times 166.667 \approx 166.167$.

Split $100+100$**:** First: fee-free $100 \times 1000/1100 \approx 90.909$. Trader receives $\text{out}_1 = 0.997 \times 90.909 \approx 90.636$. $Y_1 = 1000 - 90.636 = 909.364$. Second: fee-free $100 \times 909.364/1200 \approx 75.780$. Trader receives $\text{out}_2 = 0.997 \times 75.780 \approx 75.553$. Total: $90.636 + 75.553 = 166.189$.

$\text{Out}_{\text{split}} - \text{Out}_{\text{one}} \approx 0.022 > 0$: *superadditive* (split better).

Exact gap from (21): $0.997 \times 0.003 \times 100 \times 100 \times 1000/(1100 \times 1200) \approx 0.023$.

7 Comparison with the Semigroup Construction

The companion paper [PeggedSwap, 2026] derives that strict additivity with reinvested fees requires the liquidity update to satisfy the *semigroup law*:

$$\Gamma(\Gamma(D, a), b) = \Gamma(D, a + b) \qquad \forall\, D > 0,\ a, b \geq 0. \tag{22}$$

The current `GrowLiquidity` implementation computes L from reserves:

$$L_{\text{new}} = \Big(\prod_i X_i'\Big)^{1/n},$$

where X_i' are the post-swap concentrated balances. This is *not* a deterministic function of (L_{old}, Δ) alone—it depends on the swap outcome, which depends on the current price (ratio of reserves). Therefore it cannot satisfy the semigroup law (22) when fees are present.

Aspect	**Current implementation**	**Semigroup construction**
L update	$L_{\text{new}} = (\prod X_i')^{1/n}$ (from reserves)	$L_{\text{new}} = L + f\Delta$ (deterministic)
Depends on swap outcome?	Yes	No
With input-fee	Subadditive (safe)	—
With output-fee	Superadditive (exploitable)	—
With semigroup fee	—	Strictly additive (ideal)
Semigroup law?	$\Gamma(\Gamma(D,a),b) \neq \Gamma(D,a+b)$ in general	$\Gamma(\Gamma(D,a),b) = \Gamma(D,a+b)$ by construction

8 Discussion and Practical Implications

8.1 Safety of the Current Implementation

The standard `AquaAMM` program uses **input fees** (`_flatFeeAmountInXD`). Under this configuration, the mechanism is *subadditive*: one-shot swaps yield at least as much output as split swaps. This is safe because:

(a) Traders cannot extract more by splitting trades.

(b) MEV bots gain no advantage from chunking.

(c) The protocol collects fees regardless of execution path.

8.2 Danger of Output Fees

When output fees (`_flatFeeAmountOutXD`) are used, the mechanism becomes *superadditive*: splitting yields strictly more output. This creates an arbitrage opportunity that can be exploited. The existing test suite correctly skips additivity assertions for output-fee configurations (`skipAdditivity = true` in `ConcentrateXYCFeesInvariants`).

8.3 Path to Strict Additivity

To achieve strict additivity with reinvested fees (the ideal case), the implementation would need to replace the reserve-derived L update with a deterministic semigroup rule:

$$L_{\text{new}} = L_{\text{old}} + f \cdot \Delta \qquad \text{(additive semigroup)},$$

or

$$L_{\text{new}} = L_{\text{old}} \cdot e^{\lambda\Delta} \qquad \text{(multiplicative semigroup)}.$$

Under either rule, pool growth is deterministic and proportional to traded volume, independent of execution chunking.

9 Summary

- **Fee-free GrowLiquidity is strictly additive** because the constant-product invariant $K = XY$ is preserved (Theorem 1).
- **Input-fee GrowLiquidity is subadditive** (one-shot $\geq$ split): the fee injects extra input-token reserves early, making subsequent chunks less favorable (Theorem 2). This is the safe and standard configuration.
- **Output-fee GrowLiquidity is superadditive** (split $>$ one-shot): the fee keeps extra output-token reserves, making subsequent chunks more favorable (Theorem 3). The exact gap is $\frac{(1-f)f\,abY}{(X+a)(X+a+b)}$. This is exploitable and should be avoided.
- **The implementation does not satisfy the semigroup law** when fees are present. For strict additivity with fees, a semigroup L-update would be needed (per [PeggedSwap, 2026], Section 6).

Configuration	**Additivity**	**Comparison**	**Safe?**
GrowLiquidity (no fee)	Strictly additive	$\mathrm{Out}(a+b) = \mathrm{Out}_{\mathrm{split}}$	Yes
GrowLiquidity + input-fee	Subadditive	$\mathrm{Out}(a+b) > \mathrm{Out}_{\mathrm{split}}$	Yes
GrowLiquidity + output-fee	Superadditive	$\mathrm{Out}(a+b) < \mathrm{Out}_{\mathrm{split}}$	**No**
Semigroup L-update	Strictly additive	$\mathrm{Out}(a+b) = \mathrm{Out}_{\mathrm{split}}$	Yes (ideal)

References

PeggedSwap (2-token StableSwap) Additivity of Swaps: Fee-free strict additivity, Superadditivity of the original (Curve-style) output-fee rule, and A strictly-additive reinvested-fee construction on top of PeggedSwap. Technical Report, 2026.

J. Aczél. *Lectures on Functional Equations and Their Applications.* Academic Press, New York, 1966.

G. Angeris and T. Chitra. Improved price oracles: Constant function market makers. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies (AFT '20)*, pages 80–91, 2020. arXiv:2003.10001.

G. Angeris, T. Chitra, A. Evans, and S. Boyd. Optimal routing for constant function market makers. *arXiv preprint arXiv:2204.05238*, 2022.

M. Egorov. StableSwap – efficient mechanism for stablecoin liquidity. Curve Finance Whitepaper, November 2019. `https://curve.fi/files/stableswap-paper.pdf`.

J. Milionis, C. C. Moallemi, T. Roughgarden, and A. L. Zhang. Automated market making and loss-versus-rebalancing. In *Science of Blockchain Conference (SBC'22)*, 2022. arXiv:2208.06046.